

A Novel Parallel Multi-Stage Architecture for Network Intrusion Detection

Abstract

As cyber threats grow in frequency and sophistication, the need for accurate and efficient intrusion detection systems has become increasingly critical. This article proposes a novel parallel multi-stage network intrusion detection system designed to improve detection accuracy, reduce training and inference time, and enhance the detection of attacks. In the first stage, our architecture features two modules running in parallel: a one-class Support Vector Machine trained solely on benign traffic for anomaly detection, and a random forest classifier trained exclusively on malicious samples for attack classification. A lightweight rule-based component in the second stage integrates the outputs of both modules using pre-defined thresholds to issue the final verdict, distinguishing between benign traffic, known attacks, and previously unseen attacks (zero-day). Evaluated on the CIC-IDS2017 dataset, the proposed approach outperforms the state-of-the-art in terms of balanced accuracy while reducing the inference time by 27%. Ablation studies demonstrated that these thresholds, along with the rules, can be adjusted post-training to fine-tune the system's performance without requiring model retraining, offering operational flexibility.

Keywords: Cybersecurity, intrusion detection, parallel architecture, multi-stage detection, multi-class classification, binary classification, rule-based classification

1. Introduction

As the global landscape becomes increasingly integrated, computer network security has become a top priority within organisations of all sizes. Threats in cyberspace are increasing and occurring more frequently, and the price to fix data breaches has never been so high in history. In 2024, the global average cost has increased by 10% compared to the previous year,

reaching a global average cost of \$4.88 million. Notably, the United States has the highest average cost, with an astounding \$9.36 million per breach [1]. Therefore, there has never been a more vital need for robust defence measures that protect critical information and maintain operational integrity.

Intrusion Detection Systems (IDSs) are security systems designed to monitor a network for suspicious activity. IDS can be categorised as Network IDS (NIDSs) or Host-based IDS (HIDSs). The difference is that NIDSs are placed in strategic locations within a network and monitor the incoming network traffic, while HIDSs are installed on hosts and their duty is to monitor the activity within the host itself (e.g., checking for unusual system calls). It is worth noting that these systems themselves do not prevent a cyber threat; their duty is to signal to an administrator or another process that suspicious activity is occurring. IDS can be alternatively classified as Signature-based IDS (SIDS), Anomaly-based IDS (AIDS), or Hybrid IDS (a combination of SIDS and AIDS). SIDS compares network traffic or system activity against a database of known attack patterns (signatures). It effectively identifies recognised threats, but cannot detect new or unknown attacks. AIDS detects threats by identifying deviations from normal behaviour rather than relying on predefined signatures. This system is effective in discovering new types of network attacks; however, achieving this is far from trivial, as distinguishing between normal and abnormal profiles in intrusion detection remains challenging [2, 3]. Historically, rule-based systems and statistical models have primarily been used to distinguish between normal and abnormal behaviour in network traffic. However, these methods suffer from being computationally expensive or prone to high false alarm rates. Furthermore, rule-based systems lack reliability when compared with intrusion scenarios that fall outside the predefined knowledge base [4].

In recent years, with the advent and popularity of Artificial Intelligence (AI), many researchers have been working to apply or discover new Machine Learning (ML) models to distinguish between normal and malicious activity [2]. The implementation of data-driven models has significantly reduced the cost of data breaches, saving an average of \$2.2 million by shortening the time required to detect and contain the breaches, thereby minimising the resulting damage [1]. However, the most accurate models are often so large that their inference latency becomes prohibitive, a critical factor with both economic and operational security implications [5].

Our study focuses on creating NIDS by applying ML methodologies to one of the most widely used and important datasets, the CIC-IDS2017 [6].

Specifically, we introduce a parallel, two-stage framework: a One-Class SVM (OCSVM) trained exclusively on benign traffic is responsible for determining whether the network is under threat, while a Random Forest (RF) trained solely on malicious traffic infers the attack category. A rule-based module then fuses these outputs to confirm the presence of an attack and, when one is detected, to classify it as either a known intrusion or a zero-day (unknown) attack.

Our contributions are as follows:

- We develop a novel parallel multi-stage approach for NIDS that can be applied wherever fast training and inference are required in problems where new, unseen positive classes may arise.
- We introduce a model that allows less malicious traffic to bypass detection more effectively than the state-of-the-art approach.
- We develop a rule-based decision head that can be easily tuned to achieve different trade-offs, or, as demonstrated, replicate the results of another state-of-the-art method with a speed-up of $1.36\times$.
- We show that our model outperforms several state-of-the-art NIDS in key metrics, including inference time and balanced accuracy.

This article is organised as follows: Section 2 reviews the relevant literature and situates our proposal within the current state of the art. Section 3 describes the entire modelling pipeline, from data preprocessing to the final model. Section 4 presents the results, while Section 5 discusses them. Section 6 demonstrates two approaches to obtaining alternative results without retraining the model. Finally, Section 7 offers conclusions and identifies promising directions for future work.

2. Literature Review

Artificial intelligence is playing an increasingly central role in shaping modern society, with its applications extending across a wide range of fields. Researchers have developed advanced systems that support healthcare [7], enhance academic advising [8], and facilitate visual defect detection [9], underscoring its broad and growing impact. In recent years, the use of AI in the cybersecurity field has also been widely investigated, as many enterprises have sensitive data and resources that must be protected [10].

2.1. CIC-IDS2017

The CIC-IDS2017 dataset has been extensively analysed and remains one of the most widely used benchmarks in NIDS research. In [11] and [12], the authors conduct a comprehensive analysis of the dataset, identifying several issues, including feature duplication, errors in feature calculations, incorrect protocol detection, inconsistent TCP termination flows, and discrepancies between the labels reported on the official website and those in the original dataset. They introduce a tool to process PCAP files and produce an improved dataset version. Traditional ML models and a multi-layer perceptron are applied to both the original and refined datasets. More specifically, in [11], results indicate that the random forest classifier performs best on the improved dataset, achieving 99% accuracy, precision, and recall. However, overall performance gains are limited. Notably, CIC-IDS2017 continues to receive updates, with the latest in February 2024, potentially reducing the applicability of earlier analyses.

In [13], Support Vector Machines (SVM) and isolation-based active learning techniques are applied to predict the network attack based on clustered data. The best model achieved a score of 83.29%. In another study [14], binary classifiers such as Naive Bayes, Logistic Regression, RF, and XGBoost are evaluated to detect DoS/DDoS attacks. XGBoost outperformed others, achieving 99.11% accuracy and recall. However, while binary classification simplifies the detection process, it may not be sufficient for real-world network security applications, where distinguishing between different types of attacks is crucial.

Class imbalance remains a critical challenge in intrusion detection. [15] compares studies using the CSE-CIC IDS 2017 and 2018 datasets to experimentation with handling data imbalance. The article categorises machine learning approaches employed with the datasets and highlights the insufficient emphasis on bias because of class imbalance. It emphasises the need for more advanced supervised learning approaches to address such biases and enhance model efficiency. In particular, [16] addresses the unbalanced problem by using an autoencoder-conditional generative adversarial network. This model utilises an AE to generate a low-dimensional dataset (40 features) and a conditional generative adversarial network for oversampling rare cases. An RF classifier is trained on the augmented dataset, obtaining a precision of 98.46% and a recall of 93.29%. Another study [17] uses random down-sampling over benign examples and port scan attack rows for the multiclass classifier. In addition, the authors design a novel network intrusion detection

model, which is composed of a Convolutional Neural Network (CNN) model that produces spatial features to be fed to an LSTM, which predicts the type of attack. The model achieved a score of 0.99 in terms of accuracy and 0.99 in terms of recall. However, the dataset used to train and test the model is constructed from the original traffic packets, making it different from other studies. In [18], the authors combine correlation-based feature selection to create a subset of high-quality features and the BAT algorithm to remove redundancy among them, resulting in 13 features. Moreover, they address the class imbalance issue by employing an ensemble approach composed of multiple traditional machine learning classifiers, each tasked with identifying the type of attack. The resulting model achieved an accuracy of 99.9% and a detection rate of 99.9%.

Despite the promising results, none of the above-cited models [16, 17, 18] are designed to detect zero-day attacks.

2.2. Multi-Stage Models

In [19], a distributed IDS is proposed for the multi-layer network architecture of smart grids with the aim of attack classification. The system has been designed with three layers: the first layer is the Home Area Network, the second layer is the Neighbourhood Area Network, and the third layer is the Wide Area Network. Each layer has a local IDS module with the aim of performing an initial detection. This is done by deploying either SVM or Artificial Immune System (AIS) models that have been individually trained for each layer with different distributions of training data. The SVM models demonstrate superior detection accuracy compared to the AIS models. In another study [20], a three-tier IDS is built to detect compromised devices through user packets and flow validation using RFID, fuzzy logic, and deep learning. The first tier focuses on user authentication using RFID and digital signatures. Only packets from legitimate users arrive at the second tier, which integrates OpenFlow switches with a type-2 fuzzy filter to classify packets as usual, suspicious, or malicious based on extracted features. In the third tier, the suspicious packets are fed to a CNN for validation purposes. Although the results of [19, 20] are promising, their systems are not capable of detecting and classifying zero-day attacks.

In another work [21], a two-layer sequential architecture is proposed to predict the classes of attacks. In the first layer, five binary classifiers, one for each class in the dataset, are trained to determine whether a sample belongs to a particular class. Their outputs are summed, and if the total

is not equal to one, the tuple is classified as ambiguous and is forwarded to the second layer. In this last phase, the k-nearest neighbours algorithm is used to provide a final classification. In [22] and similarly in [23], a two-tier sequential classifier is designed. The authors performed feature selection using Linear Discriminant Analysis (LDA) and employed Naive Bayes in the first stage. Tuples classified as “normal” are passed to the second stage, where a modified KNN model incorporating a certainty factor is employed for refined classification. However, all these architectures [21, 22, 23] cannot detect zero-day attacks.

Unlike previous approaches, in [24], the authors propose a multi-level hybrid IDS capable of detecting zero-day attacks. This system is trained on a relatively small training set generated by modifying the K-means algorithm. The system processes each tuple through up to five sequential layers, where each layer employs either an SVM or an Extreme Learning Machine (ELM) to detect a specific type of attack. The zero-day attack detection is possible because, in the final layer, each tuple is classified as either “Normal” or “Unknown”, allowing the system to flag anomalous behaviour that does not match any known attack patterns.

In [25], a novel multi-level NIDS is developed, which incorporates a rule-based system and can be used to classify the kind of attacks. To begin with, the dataset is separated into nominal and numerical features. Anomaly detection rules are acquired through the use of nominal data. At the same time, relevant features are extracted from numerical data using the discrete wavelet transform to build an SVM model that can predict the category of attacks. Interactive visualisation tools are also used to facilitate the exploration of correlations between the original features and transformed features, thereby increasing the transparency of the results. Despite these advancements, the system remains vulnerable to zero-day attacks.

In [26], a sequential two-stage architecture that employs deep neural networks at both levels is proposed. In the first stage, the model outputs a probability score representing the likelihood that the tuple is normal or abnormal. This score is then passed to the second stage as an additional feature, enabling it to confirm whether the network is under attack and, if so, identify the specific type of attack. The model is trained in semi-supervised settings and presents a relatively fast inference time; however, it is not designed to detect zero-day attacks.

In [27], the designed architecture organises traffic analysis into two sequential tiers. In the first tier, the data are fed to a module built using

hierarchical agglomerative clustering; this unsupervised layer continuously refines a model of normal behaviour and, with a fast 1-neighbour check, filters out routine packets, allowing only potentially dangerous ones to proceed, thereby reducing computation overhead. The second tier then examines the suspect subset along two specialised routes: a feed-forward Neural Network (NN) recognises attacks whose patterns are observed during training, while a reinforcement-learning agent evaluates deviations that may signal previously unseen exploits. The architecture should be able to detect zero-day attacks; however, this capability is asserted rather than proven.

In [28], a two-stage architecture is proposed. In the first stage, a one-class SVM is responsible for filtering benign flows; thus, only malicious ones can reach the second stage. In this last stage, a self-organising map is used to divide the positive samples into clusters to determine the type of attack.

In [29], a sequential two-stage architecture is designed. In the first stage, an anomaly detection NN determines if the network is being attacked. If the network traffic is flagged as malicious, it would be forwarded to the second stage, where the multi-class classifier module would determine the type of attack, if known, or classify it as unknown. However, the training time for both the NNs required a vast amount of data and, consequently, a considerable amount of time.

Some studies investigate methods to accelerate training and inference times through parallelism in multi-stage architectures. In [30], a three-stage framework is applied to the CICIDS 2017 dataset. The first two stages are dedicated to data pre-processing and feature selection. In the last stage, an optimisation of hyperparameters for several ML algorithms is performed. Only the best ML model (RF) will be the classification model in the final model. Such an optimisation process can be done in parallel, therefore slowing down the training time. Similarly, [31] proposes a multi-stage approach with parallel hyperparameter optimisation but classifies using a fuzzy-based technique.

In [32], a sequential two-stage architecture is proposed. The first stage involves an autoencoder used for anomaly detection. If an attack is identified, the output of the first stage is sent to the second stage, where a random forest classifier is used to determine the type of attack. Although the proposed architecture is capable of detecting zero-day attacks, it is not parallel.

The work most closely related to ours is presented in [33], where the authors introduce a novel three-stage hierarchical intrusion detection framework, which has been demonstrated to be effective in detecting zero-day

attacks using the CIC-IDS-2017 and CSE-CIC-IDS-2018 datasets. The initial stage utilises an OCSVM to generate an anomaly score, which is then used to detect the presence of an attack within the network by comparing the score against a threshold. Only instances classified as positive are fed into the next stage: a multiclass classifier that predicts the type of attack. This last model is an RF trained only with malicious network traffic in a supervised setting. Instances for which the model has low confidence in the type of attack are forwarded to the third stage. To discriminate between low-confidence and high-confidence classifications, a second threshold, distinct from the one used in the first stage, is employed. In the third stage, the anomaly score is compared with another threshold to determine whether the attack is zero-day or benign. By varying the threshold, different models can be achieved. The best models achieved a weighted F1 of 0.9875, a F1 macro of 0.8231, an accuracy of 0.9834, a balanced accuracy of 0.9342, a zero-day recall of 0.8723, and an inference time of 7.882 seconds. Despite the promising results, the sequential execution of all stages creates a bottleneck in achieving low inference time, which is a critical component for real-time NIDS.

This article proposes a two-stage NIDS. In the first stage, an OCSVM acts as a binary detector, distinguishing between benign and malicious traffic. At the same time, a random forest classifier simultaneously performs multi-class identification of specific attack types. Running the two models in parallel reduces the resultant computational time, which is a limitation in previous studies. Additionally, since the OCSVM is trained only on benign samples and the random forest is trained on malicious samples, both models, which can be trained in parallel, utilise a "pure" subset and therefore have a lower total training time. The second phase then combines their score outputs in a lightweight, rule-based component that compares scores with predefined thresholds and provides the final classification decision. Notably, our approach enables us to adjust rules and thresholds to achieve different performances without retraining the models.

A comparison between our study and the current state-of-the-art NIDS is presented in Table 1.

3. Methodology

This section presents the work done for building the model.

Table 1: Comparison of NIDS Approaches in the Literature

Study	Detection	Classification	Stages	Zero-Day	Computational Reduction	Parallelism
[11]	✗	✓	1	✗	✗	✗
[13]	✓	✗	2	✗	✗	Limited
[14]	✓	✓	1	✗	✗	✗
[16]	✗	✓	1	✗	✗	✗
[17]	✗	✓	2	✗	✗	✗
[18]	✗	✓	1	✗	✗	✗
[19]	-	✓	3	✗	✗	✗
[20]	-	✓	3	✗	✗	✗
[21]	✗	✓	2	✗	✗	✗
[22, 23]	✗	✓	2	✗	✗	✗
[24]	✗	✓	5	✓	✗	✗
[25]	✓	✓	2	✗	✗	✗
[26]	✓	✓	2	✗	✗	✗
[27]	✓	✓	3	✓	-	✗
[28]	✓	✓	2	✗	✗	✗
[29]	✓	✓	2	✓	✗	✗
[30]	✓	✗	3	✗	✓	✓
[31]	✓	✗	3	✗	✓	✓
[32]	✓	✓	2	✓	✓	✗
[33]	✓	✓	3	✓	✓	✗
Ours	✓	✓	2	✓	✓	✓

3.1. The Dataset - CIC-IDS 2017

To build and test our model, we use the CIC-IDS 2017 dataset [6, 34]. We decided to use this dataset because it offers a good trade-off between wide adoption in the literature and the timeliness of its attack scenarios, which match common modern threat environments, thereby making our assessment more relevant to real deployments. For this study, we worked exclusively with the CSV files contained in the MachineLearningCSV subdirectory of the CIC-IDS 2017 release, using the version most recently updated on February 1, 2024.

The original dataset consists of eight separate CSV files, which we consolidated into a single file for our analysis. The dataset contains 2,830,743 rows, with each row described by 78 features and a corresponding label. The features are either of type "int64" or "float64".

3.2. Pre-processing

To ensure a fair comparison with the benchmark established in [33], we adopted the same data preprocessing logic, with the exception of Principal Component Analysis (PCA), which is excluded as it did not yield satisfactory results for our model. The complete preprocessing procedure is described in detail in [35]. However, we are unable to obtain their preprocessed database because the authors worked with an older version of the CIC-IDS2017 database, which was merged with the CIC-IDS2018 database

[36]. In particular, the version of the database on which this study is based has 2,830,743 rows and 79 columns.

To mitigate the curse of dimensionality, we:

- Dropped non-variant columns: 'Bwd PSH Flags', 'Bwd URG Flags', 'Fwd Avg Bytes/Bulk', 'Fwd Avg Packets/Bulk', 'Fwd Avg Bulk Rate', 'Bwd Avg Bytes/Bulk', 'Bwd Avg Packets/Bulk', and 'Bwd Avg Bulk Rate'.
- Dropped duplicated columns: 'Fwd Header Length.1', 'Subflow Fwd Packets', 'Subflow Bwd Packets', 'Fwd PSH Flags', 'Fwd URG Flags'
- Dropped the features that remained invariant for the majority of rows, exhibiting at most 5 distinct values across the dataset: 'Avg Fwd Segment Size', 'Subflow Fwd Bytes', 'ECE Flag Count'
- Removed the 'Destination Port' feature

Moreover, 596,780 rows are removed as they are duplicates, and given the high number of rows in the dataset, it is decided to handle the rows with missing values by removing them. More specifically, 799 rows are removed as they contain null values. A new label, 'Attack Type', is subsequently created to group similar attacks into broader categories: various DoS attacks are combined under a single 'DoS' category, 'FTP-Patator' and 'SSH-Patator' are merged into a 'Brute Force' category, and multiple web-based attacks are consolidated into a 'Web Attack' category. To enable the evaluation of zero-day attack detection, the rare 'Infiltration' and 'Heartbleed' attacks are grouped under a new 'Unknown' category. These two types are selected due to their limited presence, only 47 instances in total, thereby helping to reduce class imbalance.

At the end of this process, we have the following class distribution:

- Benign: 1896672 samples
- DoS: 321759 samples
- Brute Force: 9150 samples
- Web Attack: 2143 samples
- Port Scan: 1956 samples

- Botnet: 1437 samples
- Unknown: 47 samples

It can be noticed that the dataset is significantly unbalanced, particularly between the benign and DoS classes. It is crucial to mitigate class imbalance since training a model with an unbalanced dataset creates biased models that are then dominated by the majority classes and inclined to perform poorly for minority classes. In network intrusion detection, for instance, this could mean failing to detect less frequent but potentially high-impact categories of attacks.

To address the significant class imbalance that persisted after aggregation, downsampling is applied to ensure each category has the same number of instances as the minority class. The classes "Unknown" and "Benign" are excluded from this process. The resulting distribution is as follows:

- DoS: 1437 samples
- Brute Force: 1437 samples
- Web Attack: 1437 samples
- Port Scan: 1437 samples
- Botnet: 1437 samples

We divide the resulting "malicious" dataset into a training set (70%, 1006 instances per class) and a test set (30%, 432 instances per class). Subsequently, the training set is further split into a validation set ($\sim 30\%$, 300 instances per class) and a final training set ($\sim 70\%$, 706 instances per class). It is worth noting that all the sets are perfectly balanced across all known attack classes. Unknown instances are added to the test set, resulting in a total of approximately 2,200 instances.

Regarding benign samples, the data are subsampled and partitioned as follows: 100,000 samples are assigned to the training set, 95,551 to the validation set, and 41,857 to the test set. It is important to note that these subsets are mutually exclusive.

3.3. The Novel Architecture

As illustrated in Figure 1, the proposed parallel architecture comprises two stages. The first stage is partitioned into two concurrent modules: in Stage 1a, a binary classifier determines whether the network is under attack by calculating an anomaly score E based on the distance of each sample from the decision boundary learned by the One-Class SVM. Samples lying within the benign region are considered normal, whereas those outside the boundary are flagged as potential attacks. In Stage 1b, a multi-class classifier identifies the specific type of attack. Both Stage 1a and Stage 1b apply confidence thresholds to produce their final decisions, which are subsequently forwarded to Stage 2 for further refinement. The second stage integrates the outputs of the first stage to determine the presence of an attack and, if one is detected, to distinguish between zero-day and known attacks. In the visualisation of Stage 2 in Figure 1, "0" denotes the benign class, "1" denotes the malicious class, "AC" represents a known attack class, and "Zero Day" indicates a zero-day attack. Moreover, τ_3 is the zero-day detection threshold and is needed to refine cases where Stage 1a flags a sample as malicious but Stage 1b cannot confidently classify it. If the anomaly score exceeds τ_3 , the sample is labelled as a zero-day attack; otherwise, it is considered benign. This helps distinguish truly novel threats from benign samples misclassified by Stage 1a.

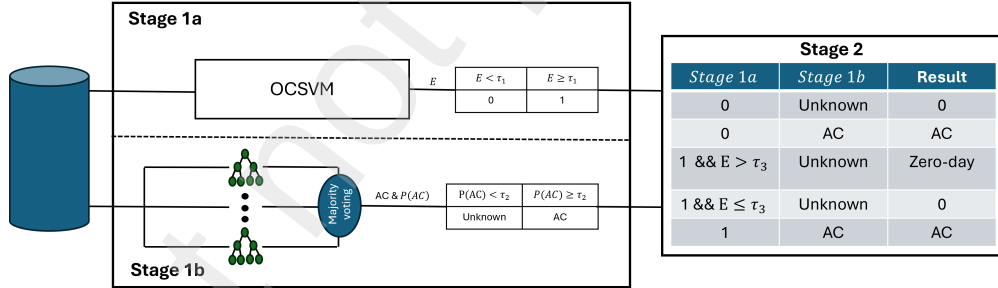


Figure 1: Proposed Novel Parallel Multi-Stage Architecture

Because each Stage-1 model is trained on a single-class dataset, the detector (Stage-1a model) is trained on benign traffic only, and the classifier (Stage-1b model) is trained on attack traffic only. Both models can be trained simultaneously, which shortens the overall training time. During inference, they also execute in parallel, so end-to-end latency is bounded by the slower of the two modules; the Stage-2 comparison step adds negligible overhead. It is worth noting that the model in Stage 1a can be substituted with any ML

algorithm able to work in an unsupervised setting. However, the multi-class classifier in Stage 1b can be substituted with any model capable of producing a probability distribution over attack categories.

The total computational complexity, together with the complexity of each individual stage of the proposed approach, is summarised in Equation 1, where T_D denotes the time complexity of the detector and T_M denotes the time complexity of the multi-class classifier.

$$\begin{aligned}
T_{\text{Stage-1a}} &= \mathcal{O}(T_D) \\
T_{\text{Stage-1b}} &= \mathcal{O}(T_M) \\
T_{\text{Stage1}} &= \mathcal{O}(\max(T_D, T_M)) \\
T_{\text{Stage2}} &= \mathcal{O}(1) \\
T_{\text{total}} &= \mathcal{O}(\max(T_D, T_M)) + \mathcal{O}(1) = \mathcal{O}(\max(T_D, T_M)) \quad (1)
\end{aligned}$$

As shown, Stage 2 introduces only a constant overhead; therefore, the overall complexity is dominated by the larger of the time complexities of the detector and the multi-class classifier.

3.3.1. Stage 1a - Binary Classifier

For the first stage, a One-Class Support Vector Machine (OCSVM) with a Radial Basis Function kernel (RBF) is adopted to model the distribution of benign network traffic and detect deviations indicative of anomalies. The OCSVM is chosen as it is well-suited for scenarios where only benign samples are reliably available during training, a common setting in network intrusion detection. By learning a compact representation of regular traffic, the model flags previously unseen or zero-day attacks as outliers without requiring explicit malicious examples. The RBF kernel is employed due to its ability to capture non-linear relationships in high-dimensional feature spaces, which are typical of network traffic data. This kernel effectively maps the input space into a higher-dimensional representation, allowing the OCSVM to form a flexible decision boundary to separate normal instances from potential anomalies.

The model is trained exclusively on benign samples to capture the intrinsic distribution of regular network traffic. To maintain computational tractability while retaining representative variability, a random subsample of 10,000 benign instances is employed. All feature sets are standardised using z-score normalisation, with the scaler fitted on the training data and

consistently applied to the validation and test sets, thereby preventing data leakage.

Once trained, the OCSVM produces anomaly scores through its decision function, which are evaluated on the validation set using multiple F_β -scores, with β ranging from 1 to 9. The decision threshold that achieved the best score among all F_β -scores is selected to provide the most effective trade-off between precision and recall. During deployment, this threshold is applied to incoming traffic to classify samples as benign or malicious, and the resulting decision is subsequently propagated to the second stage of the detection pipeline.

3.3.2. Stage 1b - Multiclass Classifier

In Stage 1b, a Random Forest classifier is employed to perform multi-class classification over known attack types. The training set consists of balanced malicious traffic samples, while the validation set is adjusted to maintain an equal proportion of benign and malicious instances for consistent threshold calibration.

The classifier outputs class probabilities for each sample, and a certainty score is derived as the maximum predicted probability across all classes. The validation set is used to analyse the certainty distributions for benign and malicious samples, enabling the selection of an optimal decision threshold that maximises the F_1 -score.

During deployment, predictions with a certainty exceeding the selected threshold are assigned their corresponding class label. In contrast, low-confidence predictions are explicitly labelled as Unknown, thereby reducing the risk of misclassification and improving robustness against previously unseen attacks. This threshold-aware strategy enables the classifier to retain high-confidence decisions while rejecting uncertain samples. The resulting decision is subsequently propagated to the second stage of the detection pipeline.

3.3.3. Stage 2 - Rule-based Classification

In Stage 2, the outputs from Stage 1a and Stage 1b are integrated to reach a final verdict. This stage plays a critical role in accurately detecting zero-day attacks. It resolves the limitations of relying on either Stage 1a or Stage 1b alone. Stage 1a, based on an OCSVM, excels at detecting anomalies—including zero-day attacks—by flagging deviations from learned benign patterns. However, it cannot provide specific information about the nature

of the attack. On the other hand, Stage 1b can classify attack types but cannot reliably recognise benign traffic; it will either label benign instances as "unknown" or misclassify them as attacks. Five cases are identified; the rule-based system is presented in Table 2. If Stage 1a classifies a sample as benign, it is directly labelled as benign. If Stage 1a classifies an attack and Stage 1b confidently identifies a known attack class with a probability exceeding the threshold τ_2 , the corresponding attack label is assigned. This rule design is motivated by the nature of NIDS, where recall is crucial. However, when Stage 1a flags a sample as malicious but Stage 1b cannot confidently classify it, an additional refinement is performed using the zero-day detection threshold τ_3 . If the anomaly score exceeds τ_3 , the sample is labelled as a zero-day attack, indicating a previously unseen threat; otherwise, it is treated as benign to reduce false alarms. This decision fusion mechanism allows Stage 2 to balance the eventually high recall from Stage 1a with selective refinement from Stage 1b, while also explicitly detecting novel attacks that do not match known categories. τ_3 is calculated as the 95 percentile of the OCSVM anomaly scores on the validation set.

Table 2: Decision rules for combining the Stage 1a and Stage 1b outputs in Stage 2.

Stage 1a (OCSVM)	Stage 1b (RF)	Final label
Benign	Unknown	Benign
Benign	Attack class	Attack class
Malicious ($E > \tau_3$)	Unknown	Zero-day Attack
Malicious ($E \leq \tau_3$)	Unknown	Benign
Malicious	Attack class	Attack class

3.4. Hardware Settings

All experiments are conducted on an MSI GF65 Thin 9SD laptop equipped with an Intel Core i7-9750H CPU @ 2.60 GHz, 16 GB of RAM (15.8 GB usable), and a 64-bit Windows operating system based on the x64 architecture.

4. Results

We evaluated the proposed architecture using key metrics, including weighted F1 and balanced accuracy. While balanced accuracy directly mitigates the

effects of class imbalance by equally weighting per-class recall, the weighted F1-score captures the precision–recall trade-offs across all classes in proportion to their support, providing a complementary view of overall correctness. Additional metrics such as AUROC, AUPR, accuracy, precision, and recall are also included to provide a comprehensive view of model performance. The full model is compared with the state-of-the-art multi-stage NIDS described in [33]. Specifically, the baseline is implemented using the same configurations as those adopted for our models, with the only exception that PCA is applied during data preprocessing, as it is an integral part of their architecture. All reported training and inference times are calculated as the mean and standard deviation over 10 independent runs.

4.1. Stage 1a - Results

The model is tested with the malicious test set (2,203 samples) and the benign test set (41,857 samples) using the decision threshold that achieved the maximum score among the F_β values (the one that maximises F_9). The evaluation metrics for Stage 1a are mentioned in Table 3. The model achieved an AUROC of 0.8903 and an AUPR of 0.3099. With the selected threshold of 0.00015, it reached an accuracy of 0.6725 and a balanced accuracy of 0.8136. The precision, recall and F1-score are 0.1295, 0.9705, and 0.2286, respectively. In addition, the macro-averaged F1-score is 0.5103, and the weighted F1-score is 0.7639.

Table 3: Stage 1a - Evaluation Scores

Metric	Value
AUROC	0.8903
AUPR	0.3099
Accuracy	0.6725
Balanced Accuracy	0.8136
Precision	0.1295
Recall	0.9705
F1-Score	0.2286
F1-Score (Macro)	0.5103
F1-Score (Weighted)	0.7639
Training Time (s)	0.147 ± 0.011
Inference Time (s)	1.063 ± 0.021

Figure 2a displays the normalised confusion matrix for Stage 1a. The true labels are shown on the y-axis and the predicted labels on the x-axis. The matrix includes both the percentage and the absolute count of samples in each class. Out of 41,857 benign samples, 27,491 are correctly classified as benign, while 14,366 are misclassified as malicious. Among the 2,203 malicious samples, 2,138 are correctly identified, and 65 are incorrectly labelled as benign.

4.2. Stage 1b - Results

The model is evaluated on the malicious test set comprising 2,203 samples and the benign test set containing 41,857 samples. As illustrated in Table 4, the model achieves an accuracy of 0.9681, a balanced accuracy of 0.9619, a macro-averaged F1-score of 0.8320, and a weighted F1-score of 0.9741.

Table 4: Stage 1b - Evaluation Scores

Metric	Score
Accuracy	0.9681
Balanced Accuracy	0.9619
F1-Score (Macro)	0.8320
F1-Score (Weighted)	0.9741
Training Time (s)	1.053 $\pm$ 0.051
Inference Time (s)	1.049 $\pm$ 0.000

Figure 2b presents the confusion matrix for Stage 1b. The matrix reports both the percentage and the absolute number of samples for each true versus predicted class pair. The diagonal elements indicate correctly classified samples. Correctly classified samples of Botnet, Brute Force, DoS, Port Scan, Unknown, and Web Attack are 428 out of 431, 414 out of 431, 403 out of 431, 419 out of 432, 40,584 out of 41,904, and 407 out of 431, respectively. Misclassifications appear in non-diagonal cells, primarily involving confusion with the Unknown class.

4.3. Full Multi-Stage Approach - Results

The second stage is tested using the malicious test set (2,203 samples) and the benign test set (41,857 samples). The evaluation results for the complete multi-stage approach are summarised in Table 5. The first row presents the baseline results obtained with the state-of-the-art architecture

as described in [33]. In the second row, the results of our architecture are presented.

Our model achieved an improved balanced accuracy (0.9362 vs. 0.9216) and significantly reduced inference time by 27% ($1.116 \text{ s} \pm 0.023$ vs. $1.520 \text{ s} \pm 0.028$) compared to the baseline. While the F1-weighted score is slightly lower (0.9491 vs. 0.9543), and the zero-day recall remained identical (0.8297), demonstrating improved efficiency without sacrificing critical detection capabilities.

Table 5: Full Multi-Stage Approach - Evaluation Scores

Model	Bal. Accuracy	F1 weighted	Zero-day recall	Inference Time (s)
Baseline [33]	0.9216	0.9543	0.8297	1.520 ± 0.028
Our	0.9362	0.9491	0.8297	1.116 ± 0.023

The confusion matrix for the final hybrid model is presented in Figure 2c. The matrix displays both the percentage and the absolute count of predictions for each true-predicted class pair. The matrix includes seven classes: Benign, Botnet, Brute Force, DoS, Port Scan, Unknown, and Web Attack. The diagonal elements indicate the number of correctly classified instances for each class. The model correctly classified 92.1% (38,533) of benign samples. 428 out of 431 Botnet samples are correctly identified. For the Brute Force, DoS, Port Scan, and Web Attack classes, the correct classification rates are 96.1% (414), 93.5% (403), 97.0% (419), and 94.4% (407), respectively. The Zero Day class is correctly identified in 83.0% of cases (39 out of 47). In Figure 2d, the confusion matrix for the state-of-the-art NIDS [33] is shown. Our model outperforms the baseline in detecting all attack types, except for zero-day and PortScan attacks, where it achieved equivalent performance. Despite the notable difference in performance for detecting attacks, our system correctly classified 92.1% of the benign traffic, which is slightly lower than the 93.0% achieved by the baseline.

5. Discussions

In this section, the experimental results presented in Section 4 are discussed in detail.

5.1. Stage 1a - Discussion

The results of Stage 1a confirmed that the model effectively detects malicious traffic, achieving a high recall of 0.9705. This behaviour is intentional

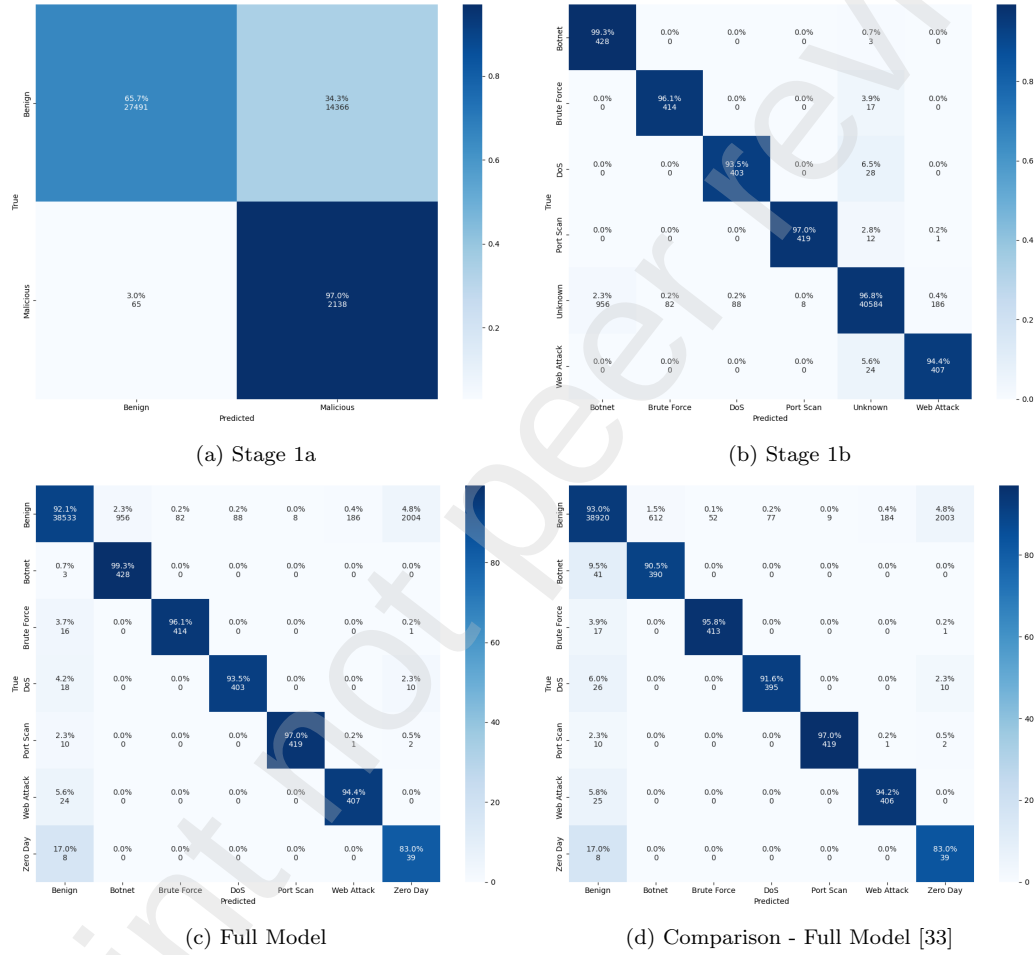


Figure 2: Confusion Matrices for Each Stage and Comparison

and aligns with the design choice of selecting the decision threshold that maximises the F_9 -score, which strongly prioritises recall over precision. In the context of network intrusion detection, it is far more critical to avoid misclassifying an attack as benign than to misclassifying a benign attack as malicious. Failing to detect malicious traffic can result in severe security breaches. Therefore, the model is deliberately tuned to minimise false negatives, even at the cost of an increased number of false positives.

As expected, this recall-oriented configuration yields a low precision of 0.1295, reflecting the higher number of benign samples incorrectly flagged as malicious. Nonetheless, the balanced accuracy of 0.8136 shows that the classifier maintains a reasonable trade-off between sensitivity to attacks and specificity towards benign traffic. Furthermore, the macro-averaged F1-score of 0.5103 and weighted F1-score of 0.7639 confirm stable performance despite the strong class imbalance in the test set.

From a ranking perspective, the AUROC of 0.8903 and AUPR of 0.3099 indicate good discriminative capability when varying the threshold. Figure 2a illustrates that 2,138 out of 2,203 malicious samples are correctly detected, while only 65 are missed. Conversely, 14,366 of the 41,857 benign samples are incorrectly flagged as malicious—an acceptable outcome in this security-sensitive context, where false alarms are preferable to undetected attacks. However, a portion of these misclassified samples are recovered in Stage 2 through the application of the zero-day threshold, which refines the decision boundaries and reduces unnecessary false positives.

In terms of computational efficiency, the model requires only 0.147 ± 0.011 s for training and 1.063 ± 0.021 s for inference, making it suitable as an initial filtering stage. By favouring recall, Stage 1a acts as a safety net, ensuring that nearly all malicious traffic is flagged for further analysis, while later stages can refine the predictions to reduce false positives.

It is worth noting that, although the selected method tends to label many samples as malicious, Stage 2 leverages the zero-day detection threshold to refine these decisions, distinguishing truly novel attacks from benign traffic misclassified by Stage 1a.

5.2. Stage 1b - Discussion

The performance of Stage 1b demonstrates that the Random Forest classifier is highly effective at distinguishing between known attack categories and benign traffic. The model achieves a high overall accuracy of 0.9681 and a balanced accuracy of 0.9619, indicating consistent performance across both

majority and minority classes. Furthermore, the macro-averaged F1-score of 0.8320 reflects strong performance even across classes with fewer samples. At the same time, the weighted F1-score of 0.9741 highlights its robustness when accounting for the class imbalance present in the test set.

The confusion matrix (Figure 2b) confirms that most attacks are correctly classified with very few misclassifications. The benign traffic also exhibited high classification reliability, with 40,584 out of 41,904 Unknown samples correctly labelled. Misclassifications primarily occurred as confusion with the Unknown class, which is expected in highly imbalanced scenarios where benign samples dominate.

From a computational perspective, the Random Forest model remains efficient, requiring only 1.053 ± 0.051 s for training and 1.049 ± 0.000 s for inference, making it suitable for deployment in real-time or near real-time settings.

Overall, these results confirm that Stage 1b can reliably identify known attack types with high accuracy while maintaining computational efficiency. Misclassifications tend to be conservative, favouring the Unknown class rather than incorrectly labelling an attack as another category, which is a desirable property for a system designed to integrate with downstream refinement stages.

5.3. Full Model - Discussion

The evaluation of the full multi-stage approach demonstrates its effectiveness in improving attack detection compared to the baseline. While the previous state-of-the-art system achieved slightly better scores in some metrics, driven by the large proportion of benign samples in the dataset, our model consistently outperformed it in the most critical areas of network intrusion detection. Specifically, the proposed architecture achieves a higher balanced accuracy (0.9362 vs. 0.9216) and identical zero-day recall (0.8297 vs. 0.8297). Furthermore, the proposed architecture significantly reduces inference time (1.116 ± 0.023 s vs. 1.520 ± 0.028 s), enhancing its suitability for real-time deployment.

The confusion matrix confirms that our model achieves high detection rates for nearly all known attacks, with a substantial improvement for Botnet attacks (99.3% vs. 90.5%), while maintaining the same zero-day detection performance as the baseline. Although benign samples are correctly classified at a slightly lower rate (92.1% vs. 93.0%), this trade-off is justified by the overall gain in attack recall.

Despite the remarkable results, the baseline performs better in terms of weighted F1-score. However, this is mainly a consequence of the strong class imbalance in the dataset, which reflects real-world traffic where benign flows vastly outnumber malicious ones. Because benign samples dominate the test set, correctly classifying them disproportionately boosts the weighted F1-score. In the context of NIDS, however, it is far more critical to avoid misclassifying a malicious sample as benign—allowing an attack to bypass detection—whereas misclassifying benign traffic as malicious, though undesirable, is a more acceptable trade-off.

In summary, the multi-stage approach enhances the system’s robustness against known attacks, prioritising recall and security sensitivity over metrics biased by benign class prevalence. This design aligns with the intrinsic requirements of NIDS, where failing to detect an attack has far more severe consequences than raising additional false alarms.

6. Ablation Studies

In this section, we demonstrate that by modifying the refinement rules and thresholds, it is possible to adapt the behaviour of the system to different operational requirements without the need to retrain any of its models. In particular, we demonstrate that by modifying just the rule-based system, without retraining any model or changing the threshold, it is possible to reproduce the same results as the baseline [33] while achieving this in less inference time. This flexibility allows the architecture to balance recall and precision depending on the specific security needs of the deployment, enabling configurations that prioritise high recall for maximum protection or lower false positive rates for environments where benign traffic dominates.

6.1. Changing the thresholds

If the goal is to improve benign traffic detection at the cost of zero-day recall, without affecting the classification of known attacks, this can be achieved by adjusting the zero-day detection threshold—for example, setting it to the 99th percentile of the validation set ($\tau_3 = 0.000849$). With this configuration, the model reached a balanced accuracy of 0.9233, a weighted F1-score of 0.9681, and a zero-day recall of 0.7021. The corresponding confusion matrix is shown in Figure 3.

As can be seen, compared to our other proposed solution, this configuration achieved a higher weighted F1-score of 0.9681 and improved benign

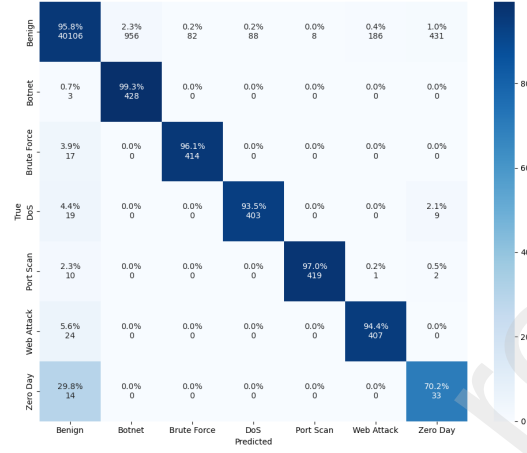


Figure 3: Confusion matrix after setting the zero-day threshold to the 99th percentile.

traffic detection by 4% without affecting the detection of known attacks. This is done at the expense of zero-day recall and balanced accuracy. Similarly, other thresholds can be tuned to obtain custom trade-offs without retraining any of the models.

6.2. Recreating the state-of-the-art results

The rule-based logic of the proposed model can be adjusted to closely replicate the baseline performance presented in Section 4. Specifically, since the baseline architecture immediately classifies traffic as benign when the detector predicts it as such, this behaviour can be simulated in our system by modifying the rule-based module to enforce a benign final prediction, regardless of the Stage 1b output. A visualisation of the modified rule-based system can be seen in Table 6. Through this approach, we achieve the same metric scores as the baseline reported in Table 5, and we obtain an identical confusion matrix to Figure 2d, with the sole exception that our model correctly classifies nine additional benign samples. This slight difference arises because the baseline employs PCA during preprocessing, whereas our architecture does not.

7. Conclusion and Future Work

This study introduces a novel parallel multi-stage architecture for network intrusion detection, designed to improve detection robustness while maintaining computational efficiency. This approach integrates two complementary

Table 6: Decision rules for combining the Stage 1a and Stage 1b outputs in Stage 2.

Stage 1a (OCSVM)	Stage 1b (RF)	Final label
Benign	Unknown	Benign
Benign	Attack class	Benign
Malicious ($E > \tau_3$)	Unknown	Zero-day Attack
Malicious ($E \leq \tau_3$)	Unknown	Benign
Malicious	Attack class	Attack class

components in Stage 1: an OCSVM anomaly detector focused on maximising recall to avoid undetected attacks, and an RF classifier specialised in recognising known attack categories. Stage 2 combines their outputs through a rule-based decision process enhanced with a zero-day threshold, enabling the system to refine predictions and effectively identify both known and previously unseen threats.

Extensive experiments demonstrated that the proposed architecture outperforms the state-of-the-art baseline in key aspects relevant to network security. In particular, it achieved a higher balanced accuracy (0.9362 vs. 0.9216) while reducing inference time by 27%, making it more suitable for real-time deployment. In addition, our model prioritised recall and improved attack detection compared to the state-of-the-art, which is critical in a security-sensitive context. Furthermore, we demonstrated that the zero-day threshold in Stage 2 partially mitigates false positives generated in Stage 1, thereby reducing unnecessary alarms without compromising attack recall.

Beyond its strong performance, the architecture provides flexibility: by adjusting only the refinement thresholds and rules, it can be adapted to different operational requirements without retraining. Ablation studies confirmed that such adjustments can optimise for alternative objectives, such as reducing false positives or reproducing baseline behaviour, demonstrating the system’s adaptability for diverse deployment scenarios.

For future work, several directions can be explored. First, replacing the OCSVM in Stage 1a with more advanced, lightweight unsupervised classifiers could further improve classification accuracy without increasing inference time. Second, exploring more advanced yet computationally efficient classifiers for Stage 1b—such as lightweight neural networks—could further enhance the recognition of known attacks while preserving real-time per-

formance. Finally, extending the architecture to other domains with similar characteristics—such as fraud detection or industrial anomaly detection—could validate its general applicability beyond network intrusion detection.

In summary, the proposed multi-stage approach provides a robust, efficient, and adaptable solution for network intrusion detection, striking a balance between high recall, low false negatives, and practical deployment efficiency.

Conflict of Interest

We have no conflicts of interest to disclose.

Data Availability

The dataset is publicly available.

References

- [1] IBM, Cost of a data breach report 2024, Tech. rep., IBM Corporation (July 2024).
URL <https://www.ibm.com/reports/data-breach>
- [2] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah, F. Ahmad, Network intrusion detection system: A systematic study of machine learning and deep learning approaches, *Transactions on Emerging Telecommunications Technologies* 32 (1) (2021) e4150.
- [3] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, *ACM Comput. Surv.* 41 (3) (Jul. 2009). doi:10.1145/1541880.1541882.
URL <https://doi.org/10.1145/1541880.1541882>
- [4] B. Mukherjee, L. Heberlein, K. Levitt, Network intrusion detection, *IEEE Network* 8 (3) (1994) 26–41. doi:10.1109/65.283931.
- [5] J. Han, W. Pak, Hierarchical lstm-based network intrusion detection system using hybrid classification, *Applied Sciences* 13 (5) (2023). doi:10.3390/app13053089.
URL <https://www.mdpi.com/2076-3417/13/5/3089>

- [6] C. I. for Cybersecurity, Intrusion detection evaluation dataset (cic-ids2017), <https://www.unb.ca/cic/datasets/ids-2017.html>, accessed: 2025-03-23 (2017).
- [7] R. Zese, L. Lombardo, M. De Maio, M. Tamascelli, F. Cremonini, et al., A novel cephalometric tool enhanced by ai assistance, in: CEUR WORKSHOP PROCEEDINGS, Vol. 3578, CEUR-WS, 2023, pp. 122–129.
- [8] M. Tamascelli, O. Bunch, B. Fowler, M. Taeb, A. Cohen, Academic advising chatbot powered with ai agent, in: Proceedings of the 2025 ACM Southeast Conference, ACMSE 2025, Association for Computing Machinery, New York, NY, USA, 2025, p. 195–202. doi:10.1145/3696673.3723065. URL <https://doi.org/10.1145/3696673.3723065>
- [9] S. B. Jha, R. F. Babiceanu, Deep cnn-based visual defect detection: Survey of current literature, Computers in Industry 148 (2023) 103911.
- [10] M. F. Ansari, B. Dash, P. Sharma, N. Yathiraju, The impact and limitations of artificial intelligence in cybersecurity: a literature review, International Journal of Advanced Research in Computer and Communication Engineering (2022).
- [11] A. Rosay, F. Carlier, E. Cheval, P. Leroux, From cic-ids2017 to lycos-ids2017: A corrected dataset for better performance, in: IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology, 2021, pp. 570–575.
- [12] A. Rosay, E. Cheval, F. Carlier, P. Leroux, Network intrusion detection: A comprehensive analysis of cic-ids2017, in: 8th international conference on information systems security and privacy, SCITEPRESS-Science and Technology Publications, 2022, pp. 25–36.
- [13] R. Singh, G. Srivastav, Novel framework for anomaly detection using machine learning technique on cic-ids2017 dataset, in: 2021 International Conference on Technological Advancements and Innovations (IC-TAI), IEEE, 2021, pp. 632–636.

- [14] S. Farhat, M. Abdelkader, A. Meddeb-Makhlouf, F. Zarai, Evaluation of dos/ddos attack detection with ml techniques on cic-ids2017 dataset., in: ICISSP, 2023, pp. 287–295.
- [15] S. S. Gopalan, D. Ravikumar, D. Linekar, A. Raza, M. Hasib, Balancing approaches towards ml for ids: A survey for the cse-cic ids dataset, in: 2020 International Conference on Communications, Signal Processing, and their Applications (ICCSPA), IEEE, 2021, pp. 1–6.
- [16] J. Lee, K. Park, Ae-cgan model based high performance network intrusion detection system, Applied Sciences 9 (20) (2019) 4221.
- [17] Y. Zhang, X. Chen, L. Jin, X. Wang, D. Guo, Network intrusion detection: Based on deep hierarchical network and original flow data, IEEE Access 7 (2019) 37004–37016.
- [18] Y. Zhou, G. Cheng, S. Jiang, M. Dai, Building an efficient intrusion detection system based on feature selection and ensemble classifier, Computer networks 174 (2020) 107247.
- [19] Y. Zhang, L. Wang, W. Sun, R. C. Green II, M. Alam, Distributed intrusion detection system in a multi-layer network architecture of smart grids, IEEE Transactions on Smart Grid 2 (4) (2011) 796–808.
- [20] A. Ali, M. M. Yousaf, Novel three-tier intrusion detection and prevention system in software defined network, IEEE Access 8 (2020) 109662–109676.
- [21] L. Li, Y. Yu, S. Bai, Y. Hou, X. Chen, An effective two-step intrusion detection approach based on binary classification and k -nn, IEEE Access 6 (2017) 12060–12073.
- [22] H. H. Pajouh, G. Dastghaibiyfard, S. Hashemi, Two-tier network anomaly detection model: a machine learning approach, Journal of Intelligent Information Systems 48 (2017) 61–74.
- [23] H. H. Pajouh, R. Javidan, R. Khayami, A. Dehghantanha, K.-K. R. Choo, A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in iot backbone networks, IEEE Transactions on Emerging Topics in Computing 7 (2) (2016) 314–323.

- [24] W. L. Al-Yaseen, Z. A. Othman, M. Z. A. Nazri, Multi-level hybrid support vector machine and extreme learning machine based on modified k-means for intrusion detection system, *Expert Systems with Applications* 67 (2017) 296–303.
- [25] S.-Y. Ji, B.-K. Jeong, S. Choi, D. H. Jeong, A multi-level intrusion detection method for abnormal network behaviors, *Journal of Network and Computer Applications* 62 (2016) 9–17.
- [26] F. A. Khan, A. Gumaei, A. Derhab, A. Hussain, A novel two-stage deep learning model for efficient network intrusion detection, *Ieee Access* 7 (2019) 30373–30385.
- [27] M. Sreelesh, et al., A two-tier network based intrusion detection system architecture using machine learning approach, in: *2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT)*, IEEE, 2016, pp. 42–47.
- [28] M. F. Umer, M. Sher, Y. Bi, A two-stage flow-based intrusion detection model for next-generation networks, *PloS one* 13 (1) (2018) e0180945.
- [29] Y. Abuadlla, G. Kvascev, S. Gajin, Z. Jovanovic, Flow-based anomaly intrusion detection system using two neural network stages, *Computer Science and Information Systems* 11 (2) (2014) 601–622.
- [30] M. Injadat, A. Moubayed, A. B. Nassif, A. Shami, Multi-stage optimized machine learning framework for network intrusion detection, *IEEE Transactions on Network and Service Management* 18 (2) (2020) 1803–1816.
- [31] S. Venkatasubramanian, Multistage optimized fuzzy based intrusion detection protocol for nids in manet, *International journal of innovative research in technology* 8 (6) (2021) 301–311.
- [32] G. Bovenzi, G. Aceto, D. Ciunzo, V. Persico, A. Pescapé, A hierarchical hybrid intrusion detection approach in iot scenarios, in: *GLOBECOM 2020-2020 IEEE global communications conference*, IEEE, 2020, pp. 1–7.

- [33] M. Verkerken, L. D’hooge, D. Sudyana, Y.-D. Lin, T. Wauters, B. Volckaert, F. De Turck, A novel multi-stage approach for hierarchical intrusion detection, *IEEE Transactions on Network and Service Management* 20 (3) (2023) 3915–3929.
- [34] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, et al., Toward generating a new intrusion detection dataset and intrusion traffic characterization., *ICISSp* 1 (2018) (2018) 108–116.
- [35] M. Verkerken, L. D’hooge, T. Wauters, B. Volckaert, F. De Turck, Towards model generalization for intrusion detection: Unsupervised machine learning techniques, *Journal of Network and Systems Management* 30 (2022) 1–25.
- [36] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Cse-cic-ids2018 on aws, <https://www.unb.ca/cic/datasets/ids-2018.html> (2018).

A Novel Parallel Multi-Stage Architecture for Network Intrusion Detection

Michael Tamascelli^{a,b}, Riccardo Morelli^{a,c}, Shashi Bhushan Jha^{a,*}

^aDepartment of Computer Science, University of West Florida, Pensacola, FL, USA

^bDepartment of Mathematics and Computer Science, University of Ferrara, Via Ludovico Ariosto, 35, Ferrara, 44121, Italy

^cDepartment of Engineering, University of Ferrara, Via Ludovico Ariosto, 35, Ferrara, 44121, Italy

*Corresponding author: Shashi Bhushan Jha (e-mail: sjha@uwf.edu)